

RAG Book Chat

Chat with your own PDF books, grounded in their exact content

Core Specifications: Retrieval-Augmented Generation (RAG) app | Interface: Streamlit Web UI | LLM: DeepSeek-R1 (via Hugging Face) | Vector DB: ChromaDB (persisted locally)

1. Overview

RAG Book Chat allows users to upload one or more PDF books and interact via conversational plain English. Unlike generic language models constrained by static training data, every response is strictly grounded in contextual passages retrieved directly from uploaded documents, accompanied by precise page citations.

This project implements a complete local Retrieval-Augmented Generation pipeline covering document ingestion, chunking, embedding generation, vector storage, similarity search, prompt construction, and LLM inference, presented through an interactive web interface.

2. Problem It Solves

- **Domain Isolation:** General-purpose LLMs lack intrinsic knowledge of private or custom documents.
- **Context Limitations:** Directly injecting an entire book exceeds token context windows and inflates inference costs.
- **Hallucination Risks:** Ungrounded LLM outputs fabricate answers without verifiable provenance.
- **The RAG Solution:** Retrieves only relevant document chunks per query, supplies explicit contextual limits to the model, and exposes source citations for complete auditability.

3. Architecture & Data Flow

Ingestion Pipeline (Document Upload):

```
Streamlit file_uploader → PyPDFLoader (pypdf) → RecursiveCharacterTextSplitter → all-mpnet-base-v2 Embeddings → ChromaDB (Disk Persistence)
```

Query Pipeline (Question Execution):

```
Chat Input Box → MMR Search (k, fetch_k, lambda) → Context Injection into Prompt Template → DeepSeek-R1 (ChatHuggingFace) → Cited Response
```

4. Key Features

Multi-Book Library Upload multiple PDFs into a unified persistent vector database without overwriting existing collections.	Adjustable Chunking Exposes chunk size and overlap parameters in sidebar controls for document-specific performance tuning.
Tunable MMR Retriever Live sliders control Maximal Marginal Relevance parameters (k, fetch_k, lambda_mult) to balance relevance and diversity.	Cited Page Sources Expandable panels disclose exact book title, page number, and source passage for verification.

5. Technology Stack

Layer	Technology	Role
Interface & Orchestration	Streamlit, LangChain	Web UI, chat interface, state management, wiring ingestion pipelines
LLM & Embeddings	DeepSeek-R1, sentence-transformers	Inference endpoint for answers; vector conversion for semantic matching
Vector Store & Parsing	ChromaDB, pypdf	Disk-persisted vector storage and page-level PDF text extraction
Environment	python-dotenv	Manages local API authentication keys

6. Step-by-Step Operation

Library Ingestion

Files are cached temporarily, parsed page-by-page preserving filenames, split via RecursiveCharacterTextSplitter (1000 chars, 100 overlap), embedded with all-mpnet-base-v2, and stored in ChromaDB.

Question Answering

Queries undergo identical vectorization and MMR retrieval. Retrieved chunks populate strict contextual prompt templates instructing DeepSeek-R1 to respond solely from provided extracts or decline explicitly.

7. Engineering Challenge: A Real Debugging Story

Transitioning script-based pipelines to Streamlit exposed a critical crash: embedding uploaded chunks triggered a generic connection/segmentation fault error uniquely inside Streamlit worker threads, despite running successfully as a standalone Python script.

Root Cause & Fix: Streamlit executes session scripts within background threads. On macOS, PyTorch multi-threaded tensor operations inside sentence-transformers are unstable when initiated from non-main threads.

Solution: Enforce single-threaded deterministic execution prior to PyTorch imports by setting environment variables and runtime thread constraints:

```
TOKENIZERS_PARALLELISM=false OMP_NUM_THREADS=1 torch.set_num_threads(1)
```

8. Future Improvements

- Implement content-hash de-duplication to prevent re-embedding existing books.
- Enable token-by-token streaming responses for enhanced user interactivity.
- Expand support to EPUB, DOCX, and plain text formats.
- Add user authentication and multi-tenant library segmentation.
- Provide fully offline local open-weight LLM execution options.

9. Running It Locally

```
pip install -r requirements.txt streamlit run rag_app.py --server.fileWatcherType none
```

Requires a valid Hugging Face API token configured locally as `HUGGINGFACEHUB_API_TOKEN`.

Prepared by Sumit Yadav — Built with Python, LangChain, and Streamlit.